Ders 8 Genişletilmiş Notu: Mediator Kalıbı, Refactoring'e Giriş ve JUnit ile Birim Sınaması

Ders 8 Genişletilmiş Notu: Mediator Kalıbı, Refactoring'e Giriş ve JUnit ile Birim Sınaması

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Mediator Kalıbı
- 2. Mediator'ın Kazancı ve Bedeli
- 3. Refactoring Nedir?
- 4. Refactoring Nasıl ve Ne Zaman Yapılır?
- 5. Code Smell (Kod Kusuru)
- 6. Birim Sınamaları
- 7. JUnit ile Birim Sınaması
- 8. Fowler'ın Video Kiralama Örneği
- 9. Örneğin JUnit ile Sınanması
- Örnek Sorular
- Dersin Toparlaması

1. Mediator Kalıbı

Geçen ders Strategy, State, Template Method ve Visitor kalıpları işlenmişti. Bu ders, tasarım kalıpları bölümünün son kalıbı olan Mediator ile başladı; ardından refactoring konusuna geçildi.

1.1 Ad ve amaç

Mediator kelime anlamıyla "ara bulucu"dur. Hoca bunu "negotiator" (müzakereci) ile karşılaştırdı: negotiator genelde iki kişinin arasını bulmaya çalışır, mediator ise daha geniş ölçekli bir uzlaşma sağlayıcıdır.

Kalıbın amacı, bir nesne topluluğu arasındaki etkileşimi sarmalamaktır. Böylece nesneler artık birbiriyle doğrudan iletişim kurmaz; ortadaki Mediator nesnesi üzerinden dolaylı ilişki kurar. Bunun kazancı şudur: nesneler değiştikçe aralarındaki ilişkiler de hemen değişmek zorunda kalmaz. Gerekli olası değişiklikler ortadaki Mediator'da sınırlı kalır, diğer nesnelere yayılmaz.

1.2 Dörtlü Çete'nin örneği: font seçme penceresi

Dörtlü Çete bu kalıp için bir font seçme penceresi örneği verir (ekrandan, 00:12:05). Ekrandaki "Font Chooser" penceresinde en üstte örnek bir yazı ("The quick brown fox…") vardır. Altında "Family" alanı ve font listesi (avant garde, chicago, courier, helvetica, new century schoolbook, palatino, times roman, zapf dingbats) bulunur. Listeden bir font seçilir seçilmez üstteki etiket o fontla yazılmaya başlar. "Weight" için radyo düğmeleri (medium, bold, demibold) vardır; bold seçilir seçilmez yazı koyulaşır. "Slant" için (roman, italic, oblique) italic seçilince yazı hemen eğikleşir. "Size" değiştikçe yazının boyu da değişir. Kısacası alttaki ayarlarda yapılan her değişiklik otomatik olarak en üstteki etikete yansır.

Hoca bileşenlerin bazen birbirini karşılıklı dışlayabileceğini de belirtti: aynı anda hem medium hem bold seçilemez. Weight ile Slant arasında böyle bir çelişki yoktur, ama daha geniş bir senaryoda olabilirdi. Soru şudur: bu penceredeki bileşenler birbirleriyle nasıl haberleşecek? Merkezi bir haberleşme noktası konursa her bileşen değişikliğini oraya bildirir, o merkez de gerekli diğer bileşenleri ve en üstteki etiketi günceller.

Ekrandaki nesne diyagramında `aClient`, `aListBox`, `aButton` ve `anEntryField` nesnelerinin her biri bir `director` referansıyla ortadaki `aFontDialogDirector` nesnesine bağlıdır; `aFontDialog-Director` da bu nesnelere geri bağlıdır.

1.3 Örnek çözüm

Slayttaki çözüm (ekrandan, 00:15:40) iki adımdır:

- GUI nesnelerine ortak bir arayüz hazırlanır ve aralarındaki etkileşimleri genel olarak tanımlayacak başka bir soyut sınıf tasarlanır.
- Bu soyut sınıftan gerçeklemeler oluşturmak için Template Method kalıbı kullanılır.

Hoca, böyle bir framework'te bileşenler için ortak bir üst sınıf zaten büyük olasılıkla vardır, yoksa bile bileşenleri eş biçimli yönetebilmek için mutlaka bir soyut üst sınıf ya da arayüz gerekir, dedi. Diyagramın yapısı şöyledir:

```
Soyut düzey:
  DialogDirector (soyut)            <---director---  Widget (soyut)
    ShowDialog()                                       Changed()
    CreateWidgets()      (soyut)                         -> director->WidgetChanged(this)
    WidgetChanged(Widget) (soyut)

Somut düzey:
  FontDialogDirector  --list-->  ListBox    (GetSelection())
    CreateWidgets()   --field--> EntryField (SetText())
    WidgetChanged(Widget)
  (ListBox ve EntryField, Widget'tan türer)
```

Hoca ekranda soyut ve somut düzeyi bir çizgiyle ayırdı. Üstte kalıbın temeli kurulur (soyut sınıflar ya da arayüzler), altta onlardan türetilen somut sınıflar özel gereksinimleri kodlar. Üst sınıf `DialogDirector` temel işlevselliği kodlar; bu işlevselliğin bazı noktalarının alt sınıflarda değiştirilmesini istemeyiz. Bu yüzden yapılacak bazı ayrıntılı işler soyut metotlarla bildirilir, bazı somut metotlar da bu soyut metotları kullanarak temel işlerin nasıl yapılacağını kodlar. Bu, daha önce görülen Template Method kalıbıdır.

Hoca ilişkilerin yönüne dikkat çekti. Soyut düzeyde ilişki `Widget`'tan `DialogDirector`'a doğrudur: her widget kendi director'ına haber verir. Somut düzeyde ise ilişki `FontDialogDirec-tor`'dan somut widget alt sınıflarına doğrudur. Çünkü somut director, ilişkide olduğu bütün bileşenlerle iki yönlü iletişimde olmak zorundadır: onlardan bilgi almak ve onlara bir şey yaptırmak zorundadır.

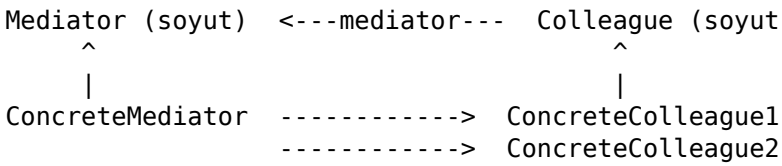
1.4 Mediator ve Observer farkı

Hoca, ilk bakışta insanın aklına Observer kalıbının gelebileceğini söyledi: `FontDialogDirec-tor` gözlemci, diğerleri gözlemlenen nesneler gibi düşünülebilir. Ama durum böyle değildir. Burada bir push ya da pull modeli yoktur. Mediator'da yapılan iş, eş bileşenler arasındaki karmaşık M×N ilişkiyi tek bir merkezi noktaya toplamaktır. Director'ın düzenlediği nesnelerden herhangi biri, herhangi bir anda bir iş yapılmasını, bir olayın ya da bir eylemler zincirinin tetiklenmesini isteyebilir. Bu, yalnızca bir varlığın durumunun değişmesinden haberdar olmak değildir.

> **Hocanın vurgusu:** Mediator ile Observer'ı karıştırmayın. Observer durum değişikliğinden haberdar olmaktır; Mediator ise eş bileşenler arasındaki M×N ilişkiyi merkezileştirir ve iş ya da eylem zinciri tetikletir. "O farkı hatırlayalım lütfen."

1.5 Kalıbın yapısı ve bileşenleri

Slayttaki kalıp yapısı (ekrandan, 00:19:10): Colleague gerçeklemeleri kendi aralarında doğrudan değil, bir Mediator gerçeklemesi üzerinden etkileşir.



Bileşenler:

- `Mediator`: Colleague nesneleriyle etkileşim kurabilecek bir arayüz sağlar.
- `ConcreteMediator`: Colleague nesneleri arasında eşgüdüm sağlar ve bunların idamesinden, yönetiminden sorumludur (sahiplik ilişkisi).
- `Colleague` / `ConcreteColleague`: etkileşimde bulunacak nesnelerin arayüzü (üstte, soyut sınıf) ve gerçeklemeleri (altta, somut sınıflar).

Kendi aralarında haberleşen bileşenlere "colleague" (meslektaş) denir. Hoca "peer" de denebileceğini, ama "colleague"in daha iyi bir terim olduğunu söyledi: bu nesnelerin yaptığı işler ve uzmanlık alanları farklı olabilir, ama bir işi yapmak için birlikte çalışırlar. Colleague'ler artık kendi aralarında haberleşmez; bütün isteklerini Mediator'a iletir. Mediator da gerekli sorgulamaları yapar ve eylemlerin yürütülmesini sağlar.

Nesneler arası etkileşim diyagramında (ekrandan, 00:20:00) beş `aColleague` nesnesinin her biri `mediator` referansıyla ortadaki `aConcreteMediator`'a bağlıdır. Hoca, Mediator'dan bütün Colleague'lere ilişkinin çift yönlü olmasının gerekmeyebileceğini belirtti. Her Colleague'in Mediator'ı bilmesi gerekir; ama somut Mediator'ın hiçbir iş yaptırmayacağı bir Colleague varsa Mediator'dan o Colleague'e doğru ilişki kurmak zorunlu değildir.

2. Mediator'ın Kazancı ve Bedeli

2.1 İlişki sayısının azalması

Hoca temel avantajı bir sayımla gösterdi. Beş Colleague birbirleriyle doğrudan etkileşseydi ve her biri diğer dördüyle eşleşebilseydi:

- Verilen: $n = 5$ nesne, tam bağlı (fully connected) etkileşim, nesnenin kendisiyle etkileşimi sayılmıyor.
- İşlem: her nesne diğer $n - 1$ nesneyle ilişkilidir, yönlü ilişki sayısı $n(n - 1)$.
- Sonuç: $5 \times 4 = 20$ ilişki.

Mediator konulunca bütün oklar çift yönlü sayılsa bile her Colleague için Mediator'a giden ve Mediator'dan gelen birer ilişki olur:

- İşlem: $2n = 2 \times 5$ (ara adım eklendi).
- Sonuç: 10 ilişki.

Böylece karmaşık ilişki ağı daha derli toplu bir hale gelir.

2.2 Bedeli

Bütün merkezi sistemlerde olduğu gibi, bu kez `ConcreteMediator`'daki olası bir hata diğer işlerin yürütülmesini karmaşıklaştırabilir ya da aksatabilir; dağıtık işler merkezi bir yere toplanmıştır. Hoca, merkezileşmenin bilinen bütün avantaj ve dezavantajlarının burada da geçerli olduğunu söyledi.

Mediator gerçeklemesi ister istemez aşırı karmaşık olur. Bu yüzden Mediator nesnelerinin başka bir projede yeniden kullanımı genelde kolay değildir. Nesneye yönelim genelde "böl ve yönet" mantığını önerir; böyle iri, ağır sınıfları pek tercih etmeyiz. Ama bu bir ödünleşimdir (trade-off): nesneler arasındaki bireysel ilişkiler fazla karmaşıksa, bu karmaşıklık merkezi noktadaki kod karmaşıklığı pahasına sadeleştirilir. Getirdikleri ve götürdükleri tartılarak kalıbın uygulanıp uygulanmayacağına karar verilir.

> **Hocanın vurgusu:** Mediator bedava değildir. İlişki ağı sadeleşir ama merkezde karmaşık, yeniden kullanımı zor bir sınıf oluşur. Kazanç ve kayıp tartılarak karar verilir.

Hoca bu kalıp için ayrı bir kod örneği yapmadı; kalıbı temel bir fikir olarak vermek istediğini söyledi. Böylece tasarım kalıpları bölümü tamamlandı.

3. Refactoring Nedir?

3.1 Bölümün amacı

Hoca şimdiye kadar tasarım kalıplarıyla çalışarak tasarım işinin nasıl iyi yapıldığını gördüklerini söyledi. Bu önemli bir öğrenme biçimidir, ama insanlar hatalarından da ders almalıdır. Bu bölümde yapılan hatalar ve bunların nasıl düzeltileceği işlenecek.

3.2 Adlandırma

Refactoring Türkçeye "yeniden yapılandırma" diye çevrilebilir, ama hoca çok oturmuş bir Türkçe karşılığı olduğunu düşünmüyor; bu yüzden derste genelde "refactoring" denecek. Hoca çeviriyi finans sektöründeki kullanımdan aldığını söyledi: orada "factoring" ile ilgili olarak borçların yeniden yapılandırılmasından söz edilir. Yapılan yanlışlara ise "code smell" denecek; hoca bunu "kod kusuru" diye çevirmeye alıştığını söyledi.

3.3 Tanım

Slayttaki tanım (ekrandan, 00:26:30):

- Bir kod parçasının ne yaptığını değiştirmeden nasıl yaptığını değiştirmeye refactoring adı verilir.
- Yazılımın dışarıdan görünen davranışı değiştirilmeden iç yapısının değiştirilmesi anlamındadır.

Yani refactoring, kodun iç yapısını iyileştirmek amacıyla yapılan değişikliklerdir. Bu sırada kodun dışarıya verdiği hizmetlerin arayüzleri, özellikle public metot imzaları değiştirilmemeye çalışılır.

3.4 Amaç: iç kalite özellikleri

Refactoring'in amacı, mevcut kod üzerinde kodun iç kalite özelliklerini iyiye götürecek değişiklikler yapmaktır. Slayttaki maddeler:

- Kodun anlaşılabilirliğinin artırılması
- Koda esneklik kazandırılması
- Yeniden kullanılabilirliğin artırılması
- Yüksek uyum ve düşük bağımlılığın sağlanması

Hoca yazılım kalitesinin iç ve dış kalite özellikleri olarak iki kısımda incelendiğini hatırlattı. İç kalite özellikleri yazılımı geliştirenleri, dış kalite özellikleri ise yazılımın kullanıcılarını ilgilendirir. Refactoring ile kodun iç kalite özellikleri iyileştirilmeye çalışılır.

Kodun anlaşılabilirliğini artıran basit etkinlikler bile refactoring sayılır ve teşvik edilir. Adı yeterince açıklayıcı olmayan bir metodun ya da üye alanın adını daha açıklayıcı yapmak bile bir refactoring eylemidir, çünkü o kod parçasının anlaşılmasını kolaylaştırır.

Esneklik kazandıran değişiklikler genelde ortaya çıkan gereksinim değişikliklerini karşılamaya yöneliktir. Yazılan kısmın başka projelerde de kullanılması hedefleniyorsa ve proje planında geride kalınmamışsa, bazı etkinlikler yeniden kullanılabilirliği iyileştirmeye ayrılabilir. Dersin ilk haftalarında konuşulan uyumu (cohesion) yükseltmek ve bağlaşımı (coupling) düşürmek de en temel iç kalite özelliklerindendir; bunları iyileştirmeye yönelik değişiklikler de refactoring'dir.

4. Refactoring Nasıl ve Ne Zaman Yapılır?

4.1 Nasıl: çevik yaklaşım

Slayttaki maddeler (ekrandan, 00:30:00):

- Çevik yaklaşımla yapılır:
  - Mevcut gereksinimleri karşılayacak ilk tasarım yapılır.
  - İlk tasarımın mükemmel olması ve her bilinmezi öngörmeye çalışması gerekmez.
  - Yeterince iyi olan tasarım gerçeklenmeye başlanır.
  - Önceki tasarım adımlarından birinde yanlış karar verildiği anlaşılınca refactoring yapılır.
- Sürekli sınama ile yapılır:
  - Yapılan değişikliklerin yeni hatalar oluşturmadığından emin olmak için her değişikliğin ardından kodun yeni hali sınanmalıdır.

Hoca elimizde zaten mevcut gereksinimleri karşılayacak bir tasarımla ortaya çıkmış bir kod olduğunu, bunu bizim ya da başkasının yazmış olabileceğini söyledi. Analiz çalışmalarının sonunda oturmuş bir gereksinimler kümesine ulaşıldıktan sonra, çok zorunlu olmadıkça değişiklik yapılmamaya çalışılır; gerekirse müşteriyle yeni bir anlaşma yapılır, süre uzatılır ya da bazı işlevlerden vazgeçilip daha önemli işlevlere odaklanılır. Yeterince iyi bir gereksinimler kümesine ulaşıldığında onu karşılayacak yeterince iyi bir tasarım yapılır ve gerçeklenir. Hatalı bir karar verildiği anlaşıldığında refactoring yapılır.

Hoca bir noktada saf refactoring yapılamayabileceğini de belirtti: yapılan değişikliklerin yazılımın dışarıya açık public metotlarını ya da arayüzlerini değiştirmesi gerekebilir. Gerekiyorsa bu iş de yapılır.

Çevik yaklaşımın temel özelliği küçük artımlardır. Aksi halde iş bütün kodu silip baştan yazmaya kadar gider. Bu yüzden her seferinde küçük bir düzeltme yaparak ilerlemek en doğrusudur.

4.2 Birim sınamalarıyla birlikte

En küçük değişiklik bile olsa doğru ve iyi yapılıp yapılmadığını anlamak için, değişikliğin etkileyebileceği kısımların testleri tekrarlanmalıdır. Bunun için en doğrusu birim sınamalarıyla çalışmaktır. Refactoring küçük adımlarla yapıldığı için her seferinde bütün testleri yeniden yürütmek gerekmez; değişiklik yapılan, dokunulan kısımların ve onlarla bir adım ilişkide olan kısımların işlevlerini sınayan test vakalarını çalıştırmak yeterlidir. Bu ölçekteki testler birim sınamalarıdır.

Değişiklik birim sınamaları eşliğinde yapılır. Yapılanın daha kötü ya da yanlış olduğu fark edilirse, testlerin ardından değişiklik hemen geri alınır. Bu bağlamda bir sürüm kontrol yazılımıyla çalışmak işi kolaylaştırır; ama derste bu kadarına girilmeyecek, yalnız testlere değinilecek.

4.3 Ne zaman

Slayttaki maddeler (ekrandan, 00:30:00):

- Kodunuz derlenip toparlanmaya gerek duyduğu zaman yapılır:
  - Kodunuza yeni özellikler eklemekte zorlanmaya başladığınızda
  - Bir hatayı bulmak için zorlanmaya başladığınız, bulduğunuzda da düzeltmek için bir çok ayrı yeri kurcalamak zorunda kaldığınızda
  - vb.
- Kodunuz kötü kokmaya başladığında yapılır!

Bu durumlar kodun esnekliğini kaybettiğini, iç kalite özelliklerinin önemli oranda düştüğünü gösterir. Böyle durumlarda refactoring'e yönelik iş paketleri tanımlamak, bir çevrim ayırmak ve yürütmek iyi olur. Bütün bunlara "kodun kötü kokmaya başlaması" denir.

5. Code Smell (Kod Kusuru)

5.1 Adlandırma

“Code smell” kelimesi kelimesine “kötü kokan kod” demektir, ama bu Türkçeye pek oturmaz. İngilizcede “it is smelly” dendiğinde bir terslik, bir aksilik, kötü bir durum olduğu anlatılır; “burada bir koku var” denir ve bu, kötü koku anlamındadır.

Hoca bunu “hatalı kod” diye çevirmenin de doğru olmadığını söyledi. Kod kusurlu olduğunda kod doğru çalışır; ama az önce bahsedilen esneklik ve bakım sorunlarına yol açar. Kod hantallaşmış, “yamalı bohçaya” dönmüştür. Çalışıyor mu? Çalışıyor. Ama yeterince iyi değil. Hata, yani bug yoktur. Bu yüzden hoca “kusurlu kod” ya da “kod kusuru” çevirisini tercih ediyor. Kod kusurları genelde kullanılan programlama yaklaşımının kusurlu kullanım örnekleridir.

> **Hocanın vurgusu:** Kod kusuru bir hata (bug) değildir. Kod çalışır ama esnekliği ve bakımı bozulmuştur.

5.2 Öğrenmede ileriye ket vurma

Hoca öğrenme psikolojisinden “ileriye ket vurma” kavramını hatırlattı: daha önce edinilen bilgi, yetenek ve alışkanlıkların yeni bir şey öğrenmeyi zorlaştırması. Örneğin uzun süre yapısal programlama yaptıktan sonra nesneye yönelime geçildiğinde, işler nesneye yönelimin en iyi uygulamalarının önerdiği şekilde değil, yapısal programlamadan kalan alışkanlıklarla yürütülebilir. Bir de “elinde çekiç olan biri her şeyi çivi olarak görür” deyişi vardır: kişi her olayda yalnız bildiği aracı kullanmaya çalışır, oysa daha doğru, daha iyi, daha yeni yöntemler olabilir. Bu yüzden kod kusurlarını bilmek, hem kendimiz yapmamak hem de yapıldığını gördüğümüzde erken önlem alıp ilgili refactoring eylemini hemen yürütmek önemli bir yetenektir.

5.3 Kod neden yamalı bohçaya döner?

Bir iş en iyi niyetle yapılmış olabilir. Ama gereksinim değişiklikleri zaman içinde parça parça gelir. Her yeni gereksinim değişikliği için bir düzeltme, ardından bir başkası yapılır. Bütün gereksinimler en başta alınabilseydi hepsini karşılayacak iyi bir tasarım yapılabilirdi. Yerel düzeltmeler üzerinden gide gide varılan nokta, genel en iyinin çok altında kalabilir. Bu yüzden kötü kokular alınmaya başlandığında refactoring yapılmalıdır.

5.4 Refactoring yeni kusur doğurabilir

Yapılan bir refactoring eylemi bir yeri düzeltirken sistemin genelinde başka kusurlar ortaya çıkarmış olabilir. Hataları (bug) test vakalarıyla anlayabiliriz; ama başka kod kusurlarına yol açıp açmadığımızı ayrıca kontrol etmemiz gerekir. Yeni bir kod kusuru çıkmışsa onun için de refactoring yapılır. Çalışma biçimi bu döngüdür.

6. Birim Sınamaları

6.1 Tanım ve tasarlanma zamanı

Slayttaki tanım (ekrandan, 00:40:00): birim sınaması (unit testing) en küçük yazılım bileşeninin sınanmasıdır; nesneye yönelik programlamada bireysel sınıfların sınanmasıdır.

Kritik soru, birim sınamalarının ne zaman tasarlanacağıdır. Üç yaklaşım vardır: kodlamadan önce (TDD), kodlama sırasında ya da kodlamanın ardından.

6.2 Test Driven Development

TDD (Test Driven Design ya da Test Driven Development), iş mantığıyla ilgili tek satır kod yazmaya başlamadan önce sınamaların tasarlanmasıdır. Hoca bunun başlı başına bir ders konusu olduğunu, hakkında kitaplar ve çalışmalar bulunduğunu söyledi. TDD savunucuları bu şekilde çalışmaya başlayanların çok memnun kaldığını ve başka türlü çalışmayı düşünmediğini söyler. Hoca bunun herkes ya da her problem için en iyi yol olmayabileceğini, ama haklı olduğu noktalar bulunduğunu belirtti.

TDD'nin haklı olduğu nokta şudur: bir şeyi düzgün sınayamıyorsanız, ya da tek başına sınayamayıp mecburen başka şeylerle birlikte sınayıp sonuçtan parça ayıklayarak tek bir bileşenin doğru çalışıp çalışmadığını anlamaya çalışıyorsanız, tasarımınız daha baştan doğru değildir. Test vakalarını hazırlarken sınama yapmada ve sonuçları değerlendirmede verimsizliklerle karşılaşıyorsanız bir tasarım hatası vardır. Bu, yazılım yaşam döngüsünde ilerlemeden önemli bir kararı erkenden alıp çok daha az masrafla değişiklik yapma fırsatı verir.

Ama yalnız bunun için TDD şart değildir. Kodlama sırasında da test vakaları hazırlanabilir: bir miktar iş mantığı yazılır, ardından yazılan kısmın sınaması yapılır. Kodlama bittikten sonra da sınama vakaları yazılabilir.

6.3 Erken sınamanın önemi

Hoca burada önemli bir noktaya dikkat çekti. Şelale modelinde (gereksinim, analiz, tasarım, kodlama, sınama adımları sırayla ve her biri tamamen bitince sonrakine geçilerek) çalışmanın dezavantajı hatırlanmalıdır. Çevik süreçlerde de "bütün kodlama bitsin, sonra sınayayım" denmez; her çevrimin sonunda o çevrimde hazırlanan kısımlar sınanır. Hatalar ne kadar geç bulunursa düzeltilmesi o kadar zorlaşır; düzeltme zorluğu ve maliyeti üstel olarak, katlanarak artar. Bu yüzden mümkün olduğunca erken hata bulabilen süreçlerle çalışmak proje başarı şansını artırır.

> **Hocanın vurgusu:** Hataları düzeltmenin maliyeti, hata ne kadar geç bulunursa o kadar katlanarak artar; bütün kodlamanın bitmesini beklemeden sınayın.

6.4 Kara kutu ve beyaz kutu sınaması

Kara kutu (black box) sınaması, sınanan bileşenin iç yapısını bilmeden ya da dikkate almadan, yalnız belirli girdi kümelerine karşılık doğru çıktıların elde edilip edilmediğine bakılarak yapılır. Beyaz kutu sınamasına oranla daha az çabayla verimli biçimde hata bulmaya yarar.

Beyaz kutu (white box) sınamasında sınanan bileşenin iç yapısı bilinir; o iç yapıdaki denetim akışının her olası dallanmasından geçecek şekilde daha fazla test vakası yazılır. Bu da yine bütün proje bittikten sonra değil, en azından hazırlanan kısım bittikten sonra yapılabilir.

6.5 Henüz yazılmamış sınıflar: vekil kod

Nesneye yönelimde sınıflar arasında ilişkiler vardır ve sınanacak sınıfın bağlaşık olduğu, ihtiyaç duyduğu başka sınıfların kodu henüz tamamlanmamış olabilir. Bu durumda sınıfımız sınanamayacak mı? Sınanabilir. Slayttaki ifadeyle (ekrandan, 00:40:00): bir sınıfın tek başına yürütemediği sorumlulukların sınanması için, bu sınıfın ihtiyaç duyduğu diğer sınıfların yerine geçecek kod gerekebilir. Bunun adları vekil, sahte, yalancı kod/sınıf vb.'dir (stub, dummy, surrogate, proxy). Vekil, yalnız ihtiyaç duyulan sınıflar gerçeklenene dek kullanılır.

Hoca bu adlar arasında nüanslar bulunduğunu, ama bunlara hâkim olacak kadar yazılım sınaması alanında olmadığını söyledi ve şu örneği verdi: karmaşık bir hesaplamanın ardından bir işlemin başarılı ya da başarısız olduğunu döndüren bir metot olsun. Bu metodu kullanan bir sınıfı sınamak için, o metoda sahip sınıfın kodunun tamamlanmasını beklemek gerekmez. Ana kod havuzunda değil ama kendi projemizde o metodu yalnız `return true` diyecek şekilde yazabiliriz. Böylece derleme hatasından kurtulur ve o hesaplamanın doğru sonuç verdiği durumlar üzerinden birim sınamalarımızı yürütürüz. Yazılmamış kısım daha sonra mutlaka yazılacaktır; tamamlandığında birim sınamalarında bir hata ortaya çıkarsa o zaman düzeltilir. Ama onun tamamlanmasını beklemek pahasına şimdiden bulunabilecek hataları bulmaktan vazgeçmemek en doğrusudur.

> **Ek bilgi (derste anlatılmadı):** Yazılım sınama literatüründe bu "test double" türleri genelde şöyle ayrılır: dummy yalnız parametre yerini doldurur ve kullanılmaz; stub önceden belirlenmiş sabit yanıtlar döndürür (dersteki `return true` örneği bir stub'dır); mock ise kendisine hangi çağrıların yapılması gerektiğini de doğrular. Bu ayrım büyük olasılıkla kaynaktan kaynağa küçük farklarla anlatılır.

Hoca konunun ayrıntıları için bölümdeki yazılım sınama teknikleri dersinin alınmasını ya da incelenmesini tavsiye etti.

7. JUnit ile Birim Sınaması

7.1 xUnit ailesi

Birim sınamaları için en sık kullanılan araç ailesi xUnit ailesidir. x yerine kullanılan dil ya da platformu simgeleyen bir iki harf konur. Java ile yazılmış kodun birim sınamaları için JUnit framework'ü vardır; C# için csUnit sürümü vardır; başka diller için de sürümleri bulunur. Slayttaki bilgiye göre (ekrandan, 00:40:00) JUnit Eclipse'te IDE ile hazır gelir (build path → add libraries), NetBeans'te de IDE ile hazır gelir. Popüler IDE'lerle birlikte gelir ve birim sınamalarını çok basit biçimde yazmayı sağlar.

Hoca JUnit'in 3.x ve 4.x sürümleri üzerinden birim sınamalarının nasıl hazırlanıp yürütüleceğini ve değerlendirileceğini anlattı.

7.2 JUnit 3.x ile test case hazırlamak

Slayttaki yapı (ekrandan, 00:50:00):

```
junit.framework paketi:
  TestCase       # setup()
  Assert         + assertEquals(beklenen, sonuc : Object)
                 + assertNotEquals(Object, Object)
                 + assertTrue(boolean)
                 + assertFalse(boolean)
                 + assertNull(Object)
                 + assertNotNull(Object)

myPackage paketi:
  AppClass1, AppClass2              (iş mantığı sınıfları)
  TestAppClass1 (+ testXXX())  --> AppClass1'i sınar, TestCase'ten türer
  TestAppClass2 (+ testYYY())  --> AppClass2'yi sınar, TestCase'ten türer
  TestAll       (+ suite() : Test) --> TestAppClass1 ve TestAppClass2'yi toplar
```

Slayttaki kurallar:

- Her sınıfın birim testi için ayrı sınıfta ayrı test case'ler hazırlamak, ardından test case'leri bir test suite altında toplamak tercih edilmiştir.
- `TestAll` sınıfını yazmak zorunlu değildir; `TestAppClass` sınıfları bireysel olarak da çalıştırılabilir.
- `TestAll.suite()` metodu statiktir.
- `TestCase.setup` metodu her `testXXX()` için yeniden çalışır.

`AppClass1`'i sınamak için birim sınamalarını barındıracak sınıfın adı `TestAppClass1` olur; sınıf adının `test` kelimesiyle başlaması bir zorunluluktur. Benzer şekilde `AppClass2` için `TestAppClass2` yazılır. Bu sınıflarda bir ya da daha fazla test metodu olur ve her metot bir test case sayılır. Metot adları neyi sınadığını açıkça anlatmalıdır; uzun olabilir, yeter ki `test` ile başlasın. Test metotları parametre almaz, almamalıdır.

Test sınıflarının otomatik çalıştırılabilmesi için iki koşul vardır: adlarının `Test` ile başlaması ve `junit.framework` paketindeki `TestCase` sınıfından kalıtımla türetilmeleri. Hoca bunun ek bir yük getirmediğini söyledi.

Test metotlarında genelde en azından kara kutu yaklaşımı kullanılır: çeşitli girdiler verilir, bir sonuç elde edilir ve elde edilen sonucun beklenen sonuç olup olmadığına bakılır.

7.3 setUp metodu

`TestCase` sınıfının protected bir `setUp()` metodu vardır. Bu metot kendi test sınıflarımızda override edilerek testlere başlamadan önce "sahne" hazırlanır. Sınanacak bir nesne çoğu zaman tek başına sınanamaz; gerekli ilklendirmelerin yapılması, ilişkilerin kurulması gerekir. Hoca not sistemi örneğini verdi: bir derste çan eğrisi uygulamasını sınamak için önce dersi oluşturmak, öğrencileri eklemek ve öğrencilere ham notlarını vermek gerekir; ancak ondan sonra çan eğrisinin doğru hesaplanıp hesaplanmadığı sınanabilir. Sınama öncesi yapılması gereken bu işler `setUp()` içine yazılır.

Her `testXXX()` metodu çalışmadan önce `setUp()` otomatik olarak yeniden yürütülür. Hoca bunun büyük olasılıkla `TestCase` sınıfında Template Method kalıbı üzerinden gerçeklendiğini söyledi.

7.4 Assert metotları

Elde edilen sonucun doğru olup olmadığını karşılaştırmak için `Assert` sınıfının public static metotları kullanılır:

- `assertEquals(beklenen, sonuc)`: iki nesne birbirine eşitse test başarılı, değilse başarısızdır.
- `assertNotEquals`: ters mantık; iki nesne eşit değilse test başarılıdır.
- `assertTrue(boolean)`: ifade `true` ise test başarılıdır. `assertFalse` bunun tersidir.
- `assertNull(Object)`: nesne `null` ise test başarılıdır. `assertNotNull` bunun tersidir.

Hoca bu metotların birbirinin yerine de kullanılabileceğini söyledi; örneğin `assertNull` yerine `assertEquals`'a beklenen değer olarak `null` verilebilir. Ama daha doğrudan bir sürümü varken dolaylı olanı kullanmaya gerek yoktur.

7.5 JUnit 4.x gelişmeleri

Slayttaki bilgiler (ekrandan, 00:56:30): geriye doğru uyumluluk korunmakla birlikte JUnit 4 sürümü ile annotation desteği gelmiştir (büyük/küçük harf duyarlılığı vardır). 3.x'e göre yazılan bütün test case'ler 4.x'te de çalışır.

- Sınama sınıflarının `TestCase` sınıfından kalıtımla türetilmesi gerekmez; ancak şu eklemeler yapılmalıdır:

```
import static org.junit.Assert.*;
import org.junit.*;
```

- Sınama metotlarının adlarını `test` kelimesiyle başlatmak gerekli değildir; ilgili metotların başına `@Test` annotation'ı koymak yeterlidir (ilk harf büyük `T`). Geriye dönük uyumluluk nedeniyle adlar yine `test` ile başlatılabilir.
- `setUp` adlı metodun yerine `@Before` annotation'ı gelmiştir. Başına `@Before` konan herhangi bir metot `setUp` mantığıyla çalışır; istenirse adı yine `setUp` kalabilir.

```
@Before
public void setUp() { /*Preparations*/ }
@Test
public void testSomething() { /*Do test*/ }
```

- Exception tanımlanan yazılımlarda atılması gereken exception'ların gerçekten ortaya çıkıp çıkmadığının sınanması da mümkün olmuştur:

```
@Test(expected=SomeException.class)
public void testTheException() throws SomeException {
    doSomethingThatCreatesTheException();
}
```

Bu testte `SomeException` oluşursa test başarılı, oluşmazsa başarısızdır. Normalde kodda bir exception oluştuğunda yürütme kesilir; her exception için testi durdurup yeniden başlatmak hiç verimli olmazdı. Bu yöntemle exception olsa da olmasa da akış kesilmez, bir sonraki testle devam edilir.

Hoca annotation'ları hatırlattı: derleyicinin göz ardı ettiği, `@` ile başlayan ve bileşenlere bilgi vermekte kullanılan satırlardır. Örneğin Eclipse'te proje ayarlarına girmeden belirli uyarıları kapatmak için `@SuppressWarnings` konur; bu derleyiciye bir şey ifade etmez ama Eclipse'e o uyarıyı göstermemesini söyler. JUnit'te de annotation'lar framework'e "bu benim test sınıfım, şu metot test case'im, şu metot setup metodum" bilgisini vermek için kullanılır.

Hoca, bir exception sınaması gerekiyorsa özellikle 4.x'i, gerekmiyorsa 3.x'i kullandığını söyledi. Ders kapsamındaki örnekler 3.x ile olacak.

8. Fowler'ın Video Kiralama Örneği

8.1 Kaynak ve anlatım sırası

Hoca refactoring kısmını, dersin ilk notlarında referansını verdiği Martin Fowler'ın "Refactoring" kitabından aldığını söyledi. Kitabı bütün ayrıntısıyla ders notlarına katmadı; kod kusurlarından ve refactoring eylemlerinden bir seçme yaptı. İlgilenenler kitaba ve konuyla ilgili başka kaynaklara bakabilir.

Fowler kitabında kod kusurlarına ve refactoring eylemlerine girmeden önce bütüncül bir örnek üzerinden bütün işleri anlatır: önce işin nasıl olduğunun hissiyatı kazandırılır, ardından ayrıntılar tek tek incelenir. Hoca bu tarzın kendisine uymadığını, önce içeriği anlatıp sonra örneklerle pekiştirmeyi tercih ettiğini söyledi ("her yiğidin bir yoğurt yiyişi vardır"). Bu yüzden ders notlarının sırası şöyledir: kusurlar, ilgili refactoring eylemleri, en sonda bütüncül örnek üzerinde öğrenilen her şeyin uygulanması. Fowler'ın kitabında her refactoring eylemi, kodun kusurlu hali bir tarafa, düzeltilmiş hali diğer tarafa konarak karşılaştırmalı anlatılır. Hoca bunu reklamlardaki "önce ve sonra" görüntülerine benzetti. Bütüncül örneğin ne olduğunu en başta konuşmak yararlı olduğu için örnek bu derste tanıtıldı; bütüncül düzeltmesi en sona bırakıldı.

8.2 Senaryo ve sınıflar

Örnek bir video kaset kiralama dükkânıdır. Hoca eskiden video kaset kiralama dükkânlarının olduğunu, sonra CD ve DVD'lerin çıktığını hatırlattı; bugün en yaygın içerik platformlarından Netflix'in de ilk başta fiziksel DVD kiralama ve postalama ile yola çıktığını söyledi.

Sınıf şeması (ekrandan, 01:20:00):

```
Movie             Rental               Customer
- _priceCode:int - _daysRented:int    + statement()

Movie 1 <---- 1 Rental * <---- 1 Customer
```

Bir müşteri (`Customer`) film (`Movie`) kiralar; kiralama işlemini `Rental` sınıfı simgeler. Hoca `Rental` sınıfının neden gerektiğini bir soruyla açıkladı: arada `Rental` olmasaydı, bir müşterinin bir filmi kaç gün kiraladığı bilgisi nerede tutulacaktı? `Movie`'de tutulsa doğru olmaz, çünkü aynı film daha sonra başka müşterilere başka sürelerle kiralanacaktır. `Customer`'da tutulsa da doğru olmaz, çünkü müşteri birden fazla film kiralayacak ve her birini farklı sayıda gün tutacaktır. Bu yaklaşımlar başlı başına bir kod kusuru olurdu. "Müşteriler filmleri kiralar" cümlesinde kiralama etkinliği de başlı başına bir varlık olarak göze çarpar; bu yüzden müşteri ile film arasında kiralama gün sayısını saklayan bir sınıf gerekir. Bir müşterinin sıfır ya da daha fazla kira kaydı vardır; her kira kaydı tek bir filmle ilişkilidir.

`Customer` sınıfındaki `statement()` metodu her kira işleminden ne kadar borç olduğunu, toplam borcu ve kazanılan bonus puanını bildirir; başka bir deyişle makbuz basar. Hoca bunun upuzun, karmaşık bir metot olduğunu ve sonraki derslerde adım adım refactor edileceğini söyledi.

`Movie` sınıfında `_priceCode` üye alanıyla simgelenen fiyat kodları vardır: normal filmler, yeni çıkan filmler ve çocuk filmleri. Bunların farklı günler için farklı kira bedelleri vardır. `Rental` sınıfı filmin kaç gün kiralandığı bilgisini tutar. Kaynak kod `refactoring/fowler00` altındadır.

8.3 statement() metodunun sıra diyagramı

Sıra diyagramında (ekrandan, 01:20:00) `Kullanıcı`, `aCustomer`, `aRental` ve `aMovie` vardır. Kullanıcı `statement()` çağırır; `aCustomer` bütün kira kayıtları için (`*[for all rentals]`) bir döngü yürütür:

- `getMovie()` ile kira kaydına hangi filmle ilişkili olduğu sorulur.
- `getPriceCode()` ile filme fiyat kodu sorulur.
- `getDaysRented()` ile kira kaydına kaç gün kiralandığı sorulur; böylece tek kira için ücret hesabı tamamlanır.
- Yine `getPriceCode()` ve `getDaysRented()` ile tek kira için bonus hesabı tamamlanır.
- `getTitle()` ile film adı alınır ve tek kira için makbuz satırı tamamlanır.

Döngü bütün kira kayıtları için tekrarlanır; sonunda biriken bilgiyle toplam borç ve toplam puan satırları eklenerek makbuz basımı tamamlanır. Hoca diyagramda `statement()` çağrısının geri dönüş okunu çizmeyi unuttuğunu belirtti ve kendine not aldı.

8.4 Customer sınıfı

`Customer` sınıfında müşterinin adı ve kira kayıtlarının tutulduğu jenerik bir liste vardır (ekrandan, 01:30:00):

```
package fowler_00;

import java.util.*;

public class Customer {
    private String _name;
    private Vector<Rental> _rentals = new Vector<Rental>();
    public Customer (String name){
        _name = name;
    }
    public void addRental(Rental arg) {
        _rentals.addElement(arg);
    }
    public String getName (){
        return _name;
    }
    ...
}
```

Hoca üye alan adlarının alt çizgiyle (_) başladığına dikkat çekti. Bu, C dilinden kalma bir alışkanlıktır, C++'ta da böyle bir gelenek vardır; Java'da normalde yapılmaz. Fowler bunu kitabında bilinçli olarak yapar: refactoring örneklerinde küçük kod parçacıkları verir ve bu parçacıklarda hangi adın üye alan, hangisinin geçici değişken olduğunu ayırt etmek gerekir. Kitap renkli olmadığı için (IDE'deki gibi üye alanların mavi görünmesi mümkün olmadığından) siyah beyaz sayfada _ ile başlayan bir ad görüldüğünde onun sınıfın üye alanı olduğu anlaşılır. Başka bir nedeni yoktur; böyle yapılması tavsiye edilmez.

Yapıcı müşterinin adını ilklendirir, `addRental` kiralama kaydını listeye ekler, `getName` adı döndürür. Ardından upuzun `statement()` metodu gelir. Hoca bunun nesneye yönelimin "böl ve yönet" mantığına ters olduğunu ve birazdan "long method" adlı kod kusuru olarak adlandırılacağını söyledi.

8.5 statement() metodunun gövdesi

Metotta toplam borcu ve toplam bonusu biriktiren iki kümülatif toplam değişkeni vardır. Kira kayıtları için bir iterator (`Enumeration`) alınır ve makbuz metni `result` adlı string'de oluşturulmaya başlanır. Ekrandaki kod (ekrandan, 01:32:20 ve 01:33:45):

```
Enumeration<Rental> rentals = _rentals.elements();
String result = "Rental Record for " + getName() + "\n";
while (rentals.hasMoreElements()) {
    double thisAmount = 0;
    Rental each = rentals.nextElement();
    //determine amounts for each line
    switch (each.getMovie().getPriceCode()) {
    case Movie.REGULAR:
        thisAmount += 2.0;
        if (each.getDaysRented() > 2)
            thisAmount += (each.getDaysRented() - 2) * 1.5;
        break;
    case Movie.NEW_RELEASE:
        thisAmount += each.getDaysRented() * 3.0;
        break;
```

```
        case Movie.CHILDRENS:
            thisAmount += 1.5;
            if (each.getDaysRented() > 3)
                thisAmount += (each.getDaysRented() - 3) * 1.5;
            break;
        }
        // add frequent renter points
        frequentRenterPoints ++;
        // add bonus for a two day new release rental
        if ((each.getMovie().getPriceCode() == Movie.NEW_RELEASE) ...
        ...
}
```

Döngüde her kira kaydı için o kaydın bedeli önce sıfırdır. Sıradaki kira kaydı alınır, ilişkili olduğu film alınır, filmin fiyat kodu öğrenilir ve bir `switch-case`'e verilir. Üç case vardır:

- Normal film (`REGULAR`): 2 lira ödenir ve bu 2 günlük kiralama hakkı verir. 2 günü geçen her gün için 1,5 lira daha ödenir.
- Yeni çıkan film (`NEW_RELEASE`): her gün için 3 lira ödenir.
- Çocuk filmi (`CHILDRENS`): 1,5 lira karşılığında 3 gün kiralama hakkı alınır; 3 günü geçen her gün için 1,5 lira daha ödenir.

Hoca tarifenin müşterinin (dükkân sahibinin) verdiği kurallara göre kodlandığını söyledi. Ardından her kiralama işlemi için bir bonus puan kazanılır. Ek olarak yeni çıkan bir film bir günden fazla kiralanmışsa bir puan daha kazanılır. Sonuç string'ine "şu film için borcunuz şu kadar" satırı eklenir ve kümülatif toplam artırılır. Döngüden çıkınca toplam borç ve kazanılan puan yazılır ve metot biter.

Hoca burada ikinci bir sorunu işaret etti: tasarım kalıplarında da söylendiği gibi, kendinizi `switch-case` ya da `if-else` kullanırken görüyorsanız bu, kalıtım ve çok biçimliliği kullanan daha iyi bir tasarım yapmanız gerektiğinin çok önemli bir göstergesidir. Düzeltme işlemlerinde buraya gelinecek.

> **Hocanın vurgusu:** `switch-case` ya da `if-else` ile tür ayırt ettiğinizi görüyorsanız, kalıtım ve çok biçimlilik kullanan daha iyi bir tasarım gerekir.

8.6 Rental ve Movie sınıfları

`Rental` sınıfı belirli bir filmle ilişkilidir ve kaç gün tutulduğu bilgisini üye alanda saklar. Yapıcı bu iki üyeyi ilklendirir; her iki üye için getter metotları vardır. Görece sade bir sınıftır.

`Movie` sınıfı (ekrandan, 01:35:20):

```
package fowler_00;

public class Movie {
    public static final int CHILDRENS = 2;
    public static final int REGULAR = 0;
    public static final int NEW_RELEASE = 1;
    private String _title;
    private int _priceCode;
    public Movie(String title, int priceCode) {
        _title = title;
        _priceCode = priceCode;
    }
    public int getPriceCode() {
        return _priceCode;
    }
    public void setPriceCode(int arg) {
        ...
    }
    ...
}
```

Film türleri sabit (`public static final int`) global değişkenlerle tanımlanmıştır: 2 çocuk filmi, 0 normal film, 1 yeni çıkan film. Hoca bunun çok doğru bir kullanım olmadığını, normalde enum kullanılacağını söyledi; bu da ileride adlandırılacak bir kusurdur. Ayrıca `_priceCode`'a bu üç değer dışında bir değer atanmasını engelleyen hiçbir şey yoktur; bir hata ya da tutarlılık kontrolü yoktur. Bir film belli bir süre sonra yeni çıkan kategorisinden normal kategorisine düşeceği için `_priceCode` `final` yapılmamış, `setPriceCode` ile değiştirilebilir bırakılmıştır.

9. Örneğin JUnit ile Sınanması

9.1 Aynı mantıksal paket, farklı klasörler

Hoca test sınıflarını uygulama mantığı sınıflarıyla aynı pakete koyduğunu söyledi. Tavsiye edilmese de uygulama sınıflarında paket düzeyinde metotlar olabilir; aynı pakette olmak bu yüzden avantaj sağlar. Java ve Eclipse, farklı fiziksel klasörlerde duran sınıfların aynı mantıksal paket altında toplanmasına izin verir. Örnekte iş mantığı sınıfları `refactoring` klasörü altındaki `fowler_00`'da, test sınıfları ise `testing` klasörü altındaki `fowler_00`'dadır; ikisi de `package fowler_00;` ile bildirilmiştir (ekrandan, 01:42:30). Alt klasör adları aynı olmak zorundadır, ama üst klasörleri farklı yerlerde olabilir.

9.2 TestMovie

`Movie` karmaşık bir sınıf değildir. Sistemde 3 film kategorisi olduğu için her kategoriden birer film oluşturulur ve türlerinin birbirinden farklı ele alınabildiği sınanır. `TestMovie` sınıfı `TestCase`'ten türetilir; bunun dışında yapıcı eklemek ya da `super` yapıcısını çağırmak gibi hiçbir şey gerekmez, yalnız test metotları eklenir. Ekrandaki kod (ekrandan, 01:42:30):

```
public class TestMovie extends TestCase {

    public void testMovie( ) {
        Movie matrix = new Movie("The Matrix",Movie.REGULAR);
        Movie monster = new Movie("Monsters, Inc.",Movie.CHILDRENS);
        Movie surrogate = new Movie("Surrogates", Movie.NEW_RELEASE);
        assertTrue("Bunlar farklı tür filmlerdir",
                matrix.getPriceCode() != monster.getPriceCode() );
        assertTrue("Bunlar farklı tür filmlerdir",
                matrix.getPriceCode() != surrogate.getPriceCode() );
        assertFalse("Bunlar farklı tür filmlerdir",
                monster.getPriceCode() == surrogate.getPriceCode() );
    }
}
```

Üç ilişkiyi ayrı ayrı test metotlarında sınamak yerine tek bir metotta üç assert kullanıldı. Bu yüzden üç assert olmasına rağmen bu tek bir test case sayılır, çünkü hepsi `test` ile başlayan tek bir metodun içindedir. Her assert ayrı metotta olsaydı üç film üye alan olarak tanımlanmalı ve `setUp()` içinde ilklendirilmeliydi. Sınıf karmaşık olmadığı için test kodu basit tutuldu; ayrıca tavsiye edilmese de bir test metodunda birden fazla assert kullanılabileceği gösterildi.

`assertTrue`'nun ilk parametresi olan mesaj, test başarısız olursa JUnit'in programcıya göstereceği açıklamadır; bu, assert metotlarının aşırı yüklenmiş (overloaded) sürümleridir ve kullanılması isteğe bağlıdır. Üçüncü karşılaştırma `assertFalse(... == ...)` ile yazılmıştır; `assertTrue(... != ...)` ile aynı kapıya çıkar.

Test çalıştırılınca JUnit penceresinde yeşil çubuk görünür: hata (error) yok, başarısızlık (failure) yok. Pencerede bir test görünür, çünkü bir test metodu vardır; içindeki üç assert ayrıca görünmez. Biri bile başarısız olsaydı kırmızı çubuk ve başarısız testin ayrıntısı görülecekti. Hoca bunu "olası en hızlı, quick and dirty test" diye niteledi.

9.3 TestCustomer: setUp

`TestCustomer` sınıfında bir müşteri ve dört film oluşturulur; bu filmler değişik sürelerle müşteriye kiralatılır ve basılan makbuzun beklendiği gibi olup olmadığı sınanır. `setUp()` içinde

nesneler yalnız oluşturulur, kiralama ilişkileri kurulmaz; çünkü her test farklı kiralama ilişkileri kuracaktır. `setUp()`'ta yapılması gereken tek şey nesnelerin ilklendirilmesidir (ekrandan, 01:45:20):

```
package fowler_00;

import junit.framework.TestCase;

public class TestCustomer extends TestCase {
    private Customer yunus;
    private Movie matrix, monster, surrogate, terminator;

    protected void setUp() {
        yunus = new Customer("Yunus Emre Selçuk");
        matrix = new Movie("The Matrix",Movie.REGULAR);
        monster = new Movie("Monsters, Inc.",Movie.CHILDRENS);
        surrogate = new Movie("Surrogates", Movie.NEW_RELEASE);
        terminator = new Movie("Terminator Salvation",Movie.NEW_RELEASE);
    }
    ...
}
```

Hoca film adlarının kodu yazdığı dönemde yeni olan filmlerden seçildiğini söyledi. `setUp()` her test metodundan önce otomatik olarak yürütülür.

9.4 TestCustomer: test metotları

`testGetName`: en basit test case, müşterinin adına doğru erişilip erişilemediğidir. Elde edilen sonuç müşterinin adıdır, beklenen çıktı `"Yunus Emre Selçuk"`'tur; ikisi `assertEquals` ile karşılaştırılır. Hoca "bu da test mi olur" denmemesi gerektiğini, çünkü adda ç gibi bir Türkçe karakter bulunduğunu ve bunun Türkçe karakterlerin doğru ele alınıp alınmadığının bir tür sınaması olduğunu söyledi.

`testStatementWhenEmpty`: hiç kiralama yapılmadan makbuz istenirse "borcunuz yok" mu denecek, yoksa bir hata sonucu borç mu yazılacak? `setUp`'ta kiralama olmadığı için müşterinin `statement()`'ı alınır; beklenen makbuz, müşterinin adıyla başlayan kira kaydı başlığı, 0 borç ve 0 puan satırlarıdır. İki string `assertEquals` ile karşılaştırılır. Hoca metot adlarında `test` ile başlayıp devamında her yeni kelimenin ilk harfini büyük yazan camel casing kuralına uyduğunu belirtti.

`testStatementWithMoviesLongRent` (ekrandan, 01:48:05):

```
public void testStatementWithMoviesLongRent( ) {
    yunus.addRental( new Rental(matrix, 3) );
    yunus.addRental( new Rental(monster, 4) );
    yunus.addRental( new Rental(surrogate, 2) );
    String sonuc = yunus.statement();
    String beklenen = "Rental Record for Yunus Emre Selçuk\n";
    beklenen += "\tThe Matrix\t3.5\n";
    beklenen += "\tMonsters, Inc.\t3.0\n";
    beklenen += "\tSurrogates\t6.0\n";
    beklenen += "Amount owed is 12.5\n";
    beklenen += "You earned 4 frequent renter points";
    assertEquals(beklenen, sonuc);
}
```

Beklenen makbuz elle hesaplanır. Adım adım:

- The Matrix (normal), 3 gün: $2.0 + (3 - 2) \times 1.5 = 3.5$
- Monsters, Inc. (çocuk), 4 gün: $1.5 + (4 - 3) \times 1.5 = 3.0$
- Surrogates (yeni), 2 gün: $2 \times 3.0 = 6.0$
- Toplam borç: $3.5 + 3.0 + 6.0 = 12.5$

- Puan: üç kiralama için 3; Surrogates yeni film ve $2 > 1$ gün kiralandığı için $+1$; toplam 4.

Hesap python ile doğrulandı; ekrandaki beklenen değerlerle aynıdır.

`testStatementWithMoviesShortRent`: üç tür film de kısa süreli kiralanır, makbuz elde edilir ve yine elle hesaplanan beklenen makbuzla karşılaştırılır. Karede beklenen makbuzun son satırları `Amount owed is 6.5` ve `You earned 3 frequent renter points`'tir (ekrandan, 01:49:05 ve 01:51:10). Hata ekranındaki beklenen değerlere göre Monsters, Inc. için 1.5, Surrogates için 3.0'dır; buradan Surrogates'in 1 gün, Monsters, Inc.'in en çok 3 gün kiralandığı ve The Matrix'in 2.0 tuttuğu (en çok 2 gün) çıkar.

`testNewReleaseRentalBonus`: bonus puan kazanma işleminin sınanmasıdır; bir normal (1 puanlık) ve bir iki puanlık kazanım için iki ayrı kira işlemi oluşturulur (ekrandan, 01:49:05):

```
public void testNewReleaseRentalBonus( ) {
    yunus.addRental( new Rental(surrogate, 2) );
    yunus.addRental( new Rental(terminator, 1) );
    String sonuc = yunus.statement();
    String beklenen = "Rental Record for Yunus Emre Selçuk\n";
    beklenen += "\tSurrogates\t6.0\n";
    beklenen += "\tTerminator Salvation\t3.0\n";
    beklenen += "Amount owed is 9.0\n";
    beklenen += "You earned 3 frequent renter points";
    assertEquals(beklenen, sonuc);
}
```

Adım adım: Surrogates 2 gün, $2 \times 3.0 = 6.0$ ve $1 + 1 = 2$ puan (yeni film, 1 günden fazla); Terminator Salvation 1 gün, $1 \times 3.0 = 3.0$ ve yalnız 1 puan (1 günden fazla değil). Toplam 9.0 ve 3 puan. Python ile doğrulandı.

`TestCustomer` çalıştırılınca JUnit penceresinde beş testin her biri ayrı ayrı görünür ve beşi de başarılıdır; yeşil çubuk çıkar.

9.5 Başarısız bir testin görünümü

Hoca bir test başarısız olsaydı ne olacağını göstermek için bir hata yaptı: müşteriyi yanlış anlamış gibi çocuk filmlerinin başlangıç ücretini 1,5 yerine 1,0 yazdı (`thisAmount += 1.0;`). Testler çalıştırılınca (ekrandan, 01:51:10) kırmızı çubuk çıktı: 5 testten 5'i çalıştı, hata (error) 0, başarısızlık (failure) 2. Başarısız testler (`testStatementWithMoviesShortRent` ve `testStatementWithMoviesLongRent`) listede işaretlendi.

Alttaki "Failure Trace" bölümünde `junit.framework.ComparisonFailure` görünür: beklenen metin (`expected:<...>`) ve elde edilen metin (`but was:<...>`) yan yana verilir ve farklılaşmanın başladığı ve bittiği yerler köşeli parantezle işaretlenir. Kısa kiralama testinde beklenen `Monsters, Inc. 1.[5 ... Amount owed is 6.5]` iken elde edilen `Monsters, Inc. 1.[0 ... Amount owed is 6.0]`'dır. Böylece iki uzun string'i tek tek karşılaştırmak gerekmez; JUnit doğrudan ilgili noktaya yönlendirir. Monsters, Inc.'in çocuk filmi olduğu görülür, koda bakılır, 1,5 olması gerekirken 1 yazıldığı fark edilir ve düzeltilir. Testler yeniden çalıştırılınca başarısız testler de başarılı olur.

Hesap kontrolü (ara adım eklendi): hatalı kodla uzun kiralamada Monsters, Inc. $1.0 + (4 - 3) \times 1.5 = 2.5$ olur ve toplam 12.0'a düşer; kısa kiralamada Monsters, Inc. 1.0, toplam $2.0 + 1.0 + 3.0 = 6.0$ olur. Bu yüzden iki test birden düşer.

9.6 Test suite: TestAll

Test sınıfları bir suite altında birleştirilebilir (ekrandan, 01:53:00):

```
package fowler_00;

import junit.framework.*;

public class TestAll {
```

```
    public static Test suite( ) {
        TestSuite suite = new TestSuite("Bütün testler");
        suite.addTestSuite(TestCustomer.class);
        suite.addTestSuite(TestMovie.class);
        return suite;
    }
}
```

Parametresiz `public static Test suite()` metodu tanımlanır, bir `TestSuite` ilklendirilir (istenirse ad verilir) ve test sınıfları `addTestSuite` ile eklenir. Parametre olarak test sınıfının `.class` bileşeni verilir. `TestAll` çalıştırılınca hem `TestCustomer` hem `TestMovie` testlerinin çalıştığı görülür; böylece test case'ler arasında çeşitli gruplamalar yapılabilir.

9.7 Reflection API

`.class` ifadesi hoca için Reflection API'ye değinme fırsatı oldu. Reflection API ile bir sınıfın kaynak kodunda hangi bileşenlerin bulunduğu öğrenilebilir: sınıfın adı, paketi, metotları, parametreleri, üye alanları. Hatta C'deki fonksiyon işaretçisine benzer şekilde belli bir sınıfın belli bir metoduna işaretçi alınıp onun üzerinden dolaylı çalıştırma gibi işler yapılabilir. Bu yetenekler Java ile birlikte gelir. Bunları birkaç adım öteye taşıyan ayrı framework'ler de vardır (bytecode mühendisliği kütüphaneleri); bunlarla bir nesnenin ait olduğu sınıfın byte kodu çalışma anında değiştirilebilir: diskte farklı bir byte kod durur, ama yüklendiğinde bellekte farklılaştırılır. Hoca bu araçları bazı yazılım metriklerini hesaplamak için kullandığını söyledi. Herhangi bir sınıf adından sonra `.class` denince Java'daki `Class` sınıfından bir nesne elde edilir ve Reflection API'nin yetenekleri bunun üzerinden kullanılır. JUnit açısından ise `addTestSuite(X.class)` ezbere yazılıp geçilebilecek bir koddur.

> **Ek bilgi (derste anlatılmadı):** Hocanın andığı byte kod mühendisliği kütüphanelerine büyük olasılıkla Apache BCEL (Byte Code Engineering Library), ASM ve Javassist örnek verilebilir. Bunlar `.class` dosyalarını okuyup çözümlemeye ve değiştirmeye yarar; bu yüzden statik kod metriklerini hesaplayan araçlarda da kullanılırlar.

9.8 Eclipse'te JUnit'i projeye eklemek

Projede JUnit ile sınama yapabilmek için proje özelliklerinde "Java Build Path" bölümüne girilir; "Libraries" sekmesinde JUnit kütüphanesinin ekli olduğu görülür. Eclipse'te başka bir projeyi kullanmak ya da jar dosyası olarak indirilmiş bir kütüphaneyi (örneğin Ant) eklemek için "Classpath" seçilip "Add External JARs" ile ilgili jar dosyası bulunur. JUnit'in kendisi için ise "Add Library" denir; "Eclipse for Java developers" sürümünde JUnit hazır gelir, seçilip "Next" ile eklenir.

Örnek Sorular

1. (derste sözlü, 00:20:23) Beş Colleague nesnesi birbiriyle tam bağlı olarak doğrudan etkileşirse kaç ilişki olur? Araya bir Mediator konunca bu sayı kaça iner ve bunun bedeli nedir?

> **Çözüm (derste verildi):** Kendisiyle etkileşim sayılmazsa her nesne diğer dördüyle ilişkilidir: $5 \times 4 = 20$ ilişki. Mediator ile bütün oklar çift yönlü sayılsa bile $2 \times 5 = 10$ ilişki kalır. Bedeli, karmaşıklığın merkezdeki `ConcreteMediator`'da toplanmasıdır: bu sınıf aşırı karmaşık olur, başka projede yeniden kullanımı zorlaşır ve içindeki bir hata bütün etkileşimi aksatabilir.

2. (derste sözlü, 00:16:41) Mediator kalıbı Observer kalıbıyla karıştırılabilir. İkisini ne ayırır?

> **Çözüm (derste verildi):** Observer'da amaç bir varlığın durumunun değişmesinden haberdar olmaktır (push ya da pull modeli). Mediator'da ise eş bileşenler arasındaki karmaşık M×N ilişki tek bir merkezi noktaya toplanır ve herhangi bir bileşen herhangi bir anda bir işin yapılmasını, bir eylemler zincirinin tetiklenmesini isteyebilir.

3. (derste sözlü, 01:21:18) Video kiralama örneğinde `Customer` ile `Movie` arasında `Rental` sınıfı olmasaydı, bir müşterinin bir filmi kaç gün kiraladığı bilgisi nerede tutulurdu ve bu neden sorunlu olurdu?

Çözüm (derste verildi): `Movie`'de tutulsa aynı film başka müşterilere başka sürelerle kiralanacağı için bilgi karışır; `Customer`'da tutulsa müşteri birden fazla filmi farklı sürelerle kiralayacağı için yine sorun çıkar. İkisi de bir kod kusuru olurdu. Kiralama başlı başına bir varlıktır; gün sayısı `Rental` sınıfında tutulur.

4. (derste sözlü, 00:44:34) Sınanacak sınıfın ihtiyaç duyduğu başka sınıfların kodu henüz tamamlanmamışsa bu sınıfın birim sınaması yapılamaz mı?

Çözüm (derste verildi): Yapılabilir. Eksik sınıfların yerine geçici bir vekil (stub, dummy, surrogate, proxy) yazılır; örneğin karmaşık hesap sonucunda başarılı/başarısız döndüren metodun gövdesine kendi projemizde `return true` yazılır. Böylece derleme hatasından kurtulunur ve sınama yürütülür. Asıl kod yazılınca sınamalar tekrarlanır.

5. (ekrandan, 01:48:05) Tarife şöyledir: normal film 2.0 (2 günden sonraki her gün +1.5), yeni film gün başına 3.0, çocuk filmi 1.5 (3 günden sonraki her gün +1.5). Her kiralama 1 puan, yeni film 1 günden fazla kiralanmışsa +1 puan. Bir müşteri The Matrix'i (normal) 3 gün, Monsters, Inc.'i (çocuk) 4 gün ve Surrogates'i (yeni) 2 gün kiralarsa toplam borcu ve puanı nedir?

Çözüm (derste verildi): The Matrix $2.0 + 1 \times 1.5 = 3.5$; Monsters, Inc. $1.5 + 1 \times 1.5 = 3.0$; Surrogates $2 \times 3.0 = 6.0$. Toplam borç 12.5. Puan $3 + 1 = 4$. Ekrandaki `testStatementWithMoviesLongRent` testi bu değerleri bekler.

6. (ekrandan, 01:51:10) Çocuk filmi başlangıç ücreti kodda yanlışlıkla 1.0 yazılırsa `TestCustomer`'daki hangi testler başarısız olur ve JUnit bunu nasıl gösterir?

Çözüm (derste verildi): Çocuk filmi içeren iki test (`testStatementWithMoviesShortRent` ve `testStatementWithMoviesLongRent`) düşer; 5 testten 2'si başarısızdır ve kırmızı çubuk çıkar. JUnit `ComparisonFailure` ile beklenen ve elde edilen metni verir, farklılaşan kısmı köşeli parantezle işaretler: kısa kiralamada beklenen `1.5` ve `6.5` yerine `1.0` ve `6.0` elde edilmiştir.

Dersin Toparlaması

Ders, tasarım kalıplarının sonuncusu olan Mediator ile başladı. Mediator, nesne topluluğu arasındaki karmaşık M×N etkileşimi tek bir merkezi nesnede toplar; Colleague'ler birbirini değil yalnız Mediator'ı bilir. Font seçme penceresi örneğinde bu yapı `DialogDirector`/`FontDialogDirector` ve `Widget` hiyerarşisiyle, Template Method yardımıyla kuruldu. Kazanç ilişki sayısının azalması, bedel ise merkezde karmaşık ve yeniden kullanımı zor bir sınıf oluşmasıdır; Observer ile karıştırılmaması gerektiği vurgulandı.

Ardından refactoring bölümüne geçildi. Refactoring, kodun dış davranışını değiştirmeden iç yapısını iyileştirmektir; çevik yaklaşımla, küçük adımlarla ve her adımdan sonra birim sınamalarıyla yapılır. Kod kötü kokmaya başladığında, yani yeni özellik eklemek ve hata ayıklamak zorlaştığında yapılır. Kod kusuru (code smell) bir hata değil, çalışan ama kusurlu koddur. Birim sınamaları, kara ve beyaz kutu yaklaşımları, vekil kod ve JUnit 3.x/4.x kullanımı anlatıldı. Son olarak Fowler'ın video kiralama örneği tanıtıldı ve JUnit testleriyle sınandı. Örnekte görülen uzun `statement()` metodu, `switch-case` ile tür ayrımı ve enum yerine `int` sabitler, sonraki konularda kod kusurları ve refactoring eylemleriyle adım adım düzeltilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.